

Airflow - First DAGs

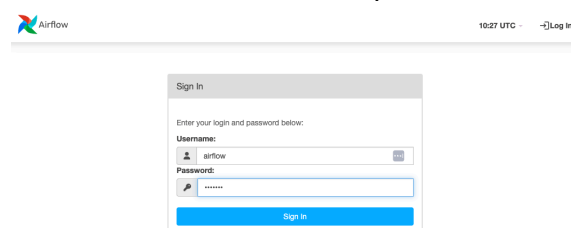
Creating an Airflow DAG:

Create **my_first_dag.py** file - define the DAG and add a PythonOperator task

```
my_first_dag.py U X
tascientest-data-engineering > airflow > dags > my_first_dag.py
1  from airflow import DAG
2  from airflow.utils.dates import days_ago
3  from airflow.operators.python import PythonOperator
4  import datetime
5
6
7  my_dag = DAG(
8      dag_id = 'my_very_first_dag',
9      description = 'My first DAG created with DataScientest',
10     tags = ['tutorial', 'datascientest'],
11     schedule_interval = None,
12     default_args = {
13         'owner': 'airflow',
14         'start_date': days_ago(2),
15     }
16 )
17
18 def print_date_and_hello():
19     print(datetime.datetime.now())
20     print('Hello from Airflow')
21
22
23 my_task = PythonOperator(
24     task_id = 'my_very_first_task',
25     python_callable = print_date_and_hello,
26     dag = my_dag,
27 )
```

Open URL <http://52.209.216.217:8080/>

Enter username airflow and password airflow



Airflow interface:

Locate DAG using filter option in Airflow UI using dag_id **my_very_first_dag**

Trigger a manual run by clicking the Play button

DAG: my_very_first_dag My first DAG created with DataScientest

Schedule: None Next Run: None

Grid Graph Calendar Task Duration Task Times Landing Times Gantt Details <> Code Audit Log

04/07/2025, 10:40:16 25 All Run Types All Run States Clear Filters Auto-refresh

Press (SHIFT) + / for shortcuts

my_very_first_dag

my_very_first_task

DAG Summary	
Total Tasks	1
PythonOperator	1

DAG Details	
Dag id	my_very_first_dag
Description	My first DAG created with DataScientest
Fileloc	/opt/airflow/dags/my_first_dag.py
Has import errors	false
Has task concurrency limits	false
Is active	true
Is rescheduled	true

Exploring dashboard of DAG my_very_first_dag

DAG: my_very_first_dag My first DAG created with DataScientest

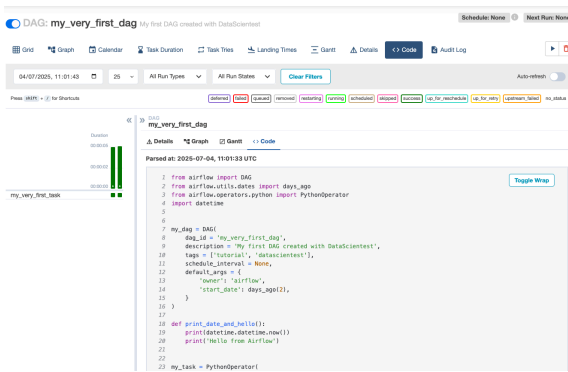
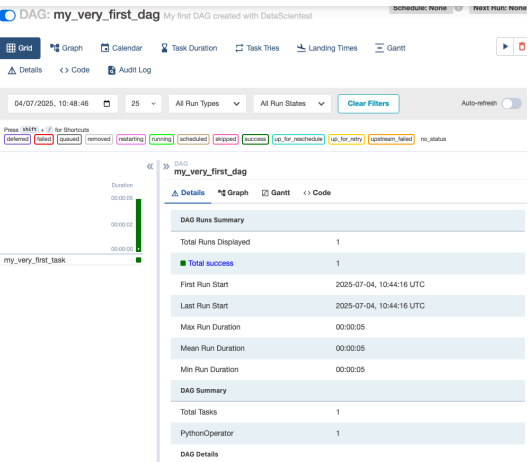
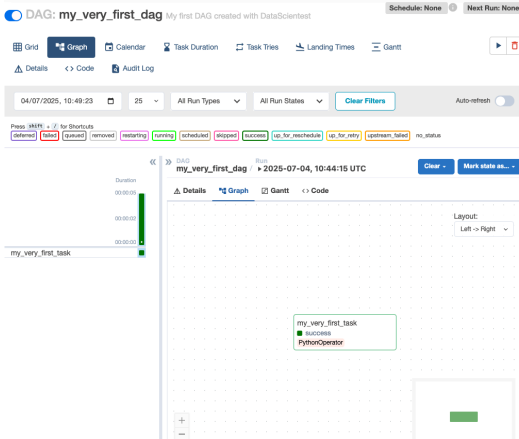
Grid Graph Calendar Task Duration Task Tries Landing Times Gantt
Details Code Audit Log

DAG Details

Success

Schedule Interval	None
Catchup	True
Start Date	None
End Date	None
Max Active Runs	0 / 16
Max Active Tasks	16
Default Args	{'owner': 'airflow', 'start_date': DateTime(2025, 7, 2, 0, 0, 0, tzinfo=Timezone('UTC'))}
Tasks Count	1
Task IDs	[my_very_first_task]
Relative file location	my_first_dag.py
Owner	airflow
Owner Links	None
DAG Run Timeout	None
Tags	datascientest tutorial

lagModel debug information



DAG: my_very_first_dag My first DAG created with DataScientest

Schedule: None Next Run: None

Grid Graph Calendar Task Duration Task Tries Landing Times Gantt
Details Code Audit Log

04/07/2025, 10:49:53 25 All Run Types All Run States Clear Filters Auto-refresh

Press (SHIFT) + / for Shortcuts
[deferred] [failed] [skipped] [removed] [retrying] [running] [scheduled] [stopped] [success] [up_for_reschedule] [up_for_retry] [upstream_failed] no_status

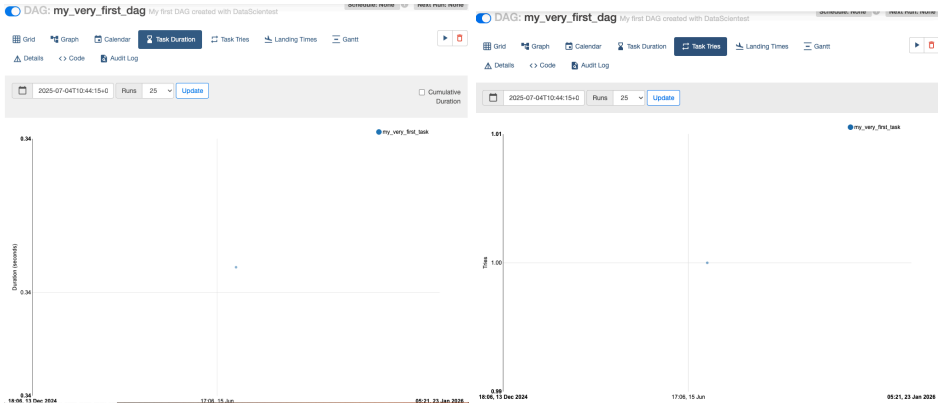
my_very_first_dag Run 2025-07-04, 10:44:15 UTC Clear Mark state as...

my_very_first_task

DAG Audit Log

This view displays selected events and operations that have been taken on this dag. The included and excluded events are set in the Airflow configuration, which by default remove view only actions. For a full list of events regarding this DAG, click here.

Time	Task ID	Event	Logical Date	Owner	Details
2025-07-04, 11:01:08	my_very_first_task	success	2025-07-04, 11:01:02.980615+02:00	airflow	None
2025-07-04, 11:01:08	my_very_first_task	cli_task_run	None	default	{'host_name': 'd77c3bc8171c', 'full_command': 'f:/home/airflow/locabin/airflow', 'oskey': 'worker'}
2025-07-04, 11:01:08	my_very_first_task	running	2025-07-04, 11:01:02.980615+02:00	airflow	None
2025-07-04, 11:01:04	my_very_first_task	cli_task_run	None	default	{'host_name': 'd77c3bc8171c', 'full_command': 'f:/home/airflow/locabin/airflow', 'oskey': 'worker'}
2025-07-04, 11:01:02	None	trigger	None	airflow	[{'origin': '/dagapi/my_very_first_dag/grid', 'dag_id': 'my_very_first_dag'}]
2025-07-04, 10:46:24	None	graph_data	None	airflow	[{'dag_id': 'my_very_first_dag'}]
2025-07-04, 10:46:28	None	graph_data	None	airflow	[{'dag_id': 'my_very_first_dag'}]
2025-07-04, 10:46:20	my_very_first_task	success	2025-07-04, 10:44:15.135622+02:00	airflow	None
2025-07-04, 10:46:24	my_very_first_task	cli_task_run	None	default	{'host_name': 'd77c3bc8171c', 'full_command': 'f:/home/airflow/locabin/airflow', 'oskey': 'worker'}
2025-07-04, 10:46:20	my_very_first_task	running	2025-07-04, 10:44:15.135622+02:00	airflow	None

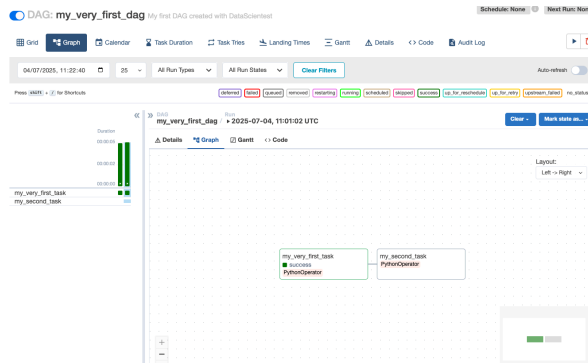


```

1 from airflow import DAG
2 from airflow.utils.dates import days_ago
3 from airflow.operators.python import PythonOperator
4 import datetime
5
6
7 my_dag = DAG(
8     dag_id = 'my_very_first_dag',
9     description = 'My first DAG created with DataScientest',
10    tags = ['tutorial', 'datascientest'],
11    schedule_interval = None,
12    default_args = {
13        'owner': 'airflow',
14        'start_date': days_ago(2),
15    }
16 )
17
18 def print_date_and_hello():
19     print(datetime.datetime.now())
20     print('Hello from Airflow')
21
22 def print_date_and_hello_again():
23     print('Hello from Airflow again')
24
25
26 my_task = PythonOperator(
27     task_id = 'my_very_first_task',
28     python_callable = print_date_and_hello,
29     dag = my_dag,
30 )
31
32 my_task2 = PythonOperator(
33     task_id = 'my_second_task',
34     python_callable = print_date_and_hello_again,
35     dag = my_dag
36 )
37
38 my_task >> my_task2

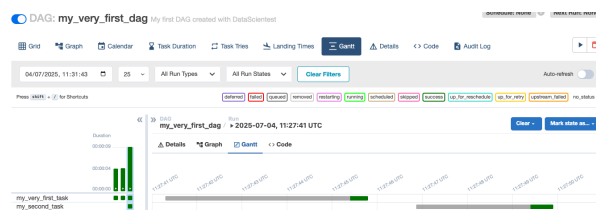
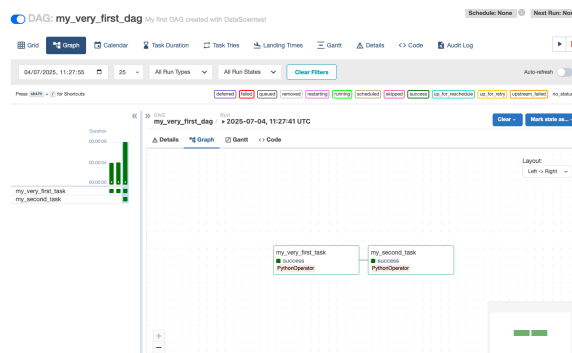
```

Return to Airflow UI and click refresh button, navigate to Graph View



Creating dependencies:

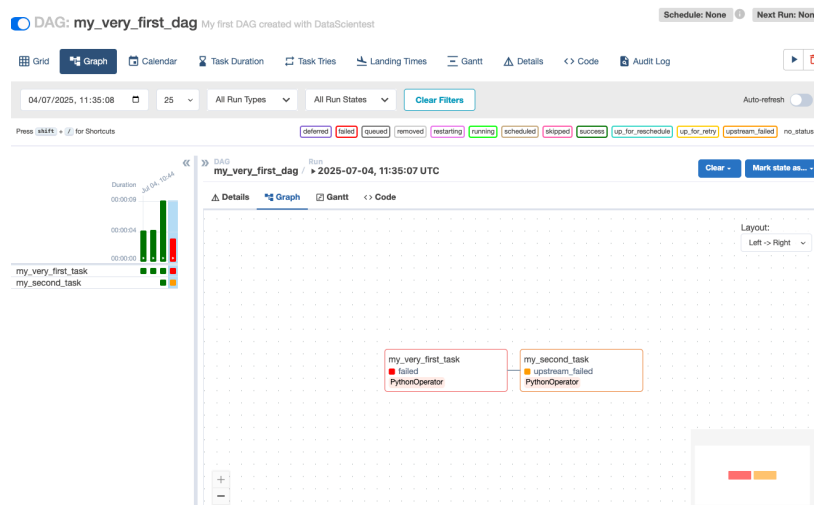
Update DAG in Airflow by clicking play button again for **my_very_first_dag** and observe updates in Graph view and Gantt tab



Edit **my_first_dag.py** file to change first task to last 30 seconds minimum

```
my_first_dag.py U X
datascientest-data-engineering > airflow > dags > my_first_dag.py
1 from airflow import DAG
2 from airflow.utils.dates import days_ago
3 from airflow.operators.python import PythonOperator
4 import datetime
5
6
7 my_dag = DAG(
8     dag_id='my_very_first_dag',
9     description='My first DAG created with DataScientest',
10    tags=['tutorial', 'datascientest'],
11    schedule_interval=None,
12    default_args={
13        'owner': 'airflow',
14        'start_date': days_ago(2),
15    }
16 )
17
18 def print_date_and_hello():
19     time.sleep(30)
20     print(datetime.datetime.now())
21     print('Hello from Airflow')
22
23 def print_date_and_hello_again():
24     print('Hello from Airflow again')
25
26
27 my_task = PythonOperator(
28     task_id='my_very_first_task',
29     python_callable=print_date_and_hello,
30     dag=my_dag,
31 )
32
33 my_task2 = PythonOperator(
34     task_id='my_second_task',
35     python_callable=print_date_and_hello_again,
36     dag=my_dag
37 )
38
39 my_task >> my_task2
```

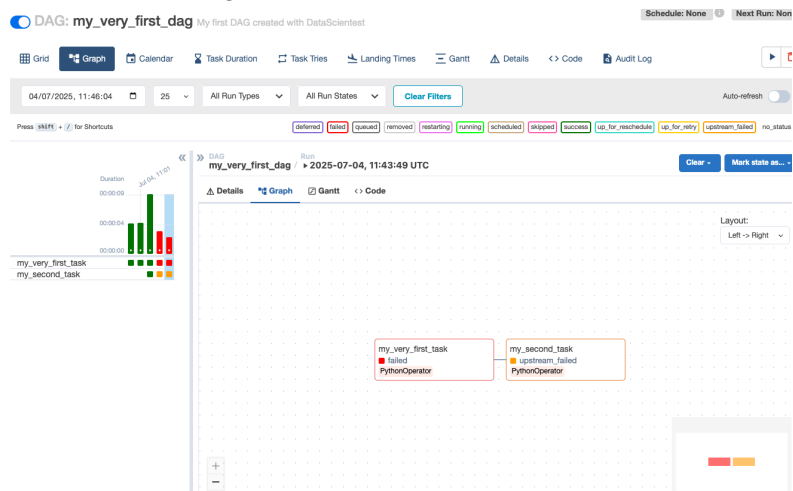
Click the play button again to update Airflow DAG and observe the latest 30 secs sleep update in the Graph view tab. The first task takes a while to run and the second task is placed on hold.



Edit **my_first_dag** file to modify first task in order to throw `TypeError` exception

```
my_first_dag.py U X
datascientest-data-engineering > airflow > dags > my_first_dag.py
1 from airflow import DAG
2 from airflow.utils.dates import days_ago
3 from airflow.operators.python import PythonOperator
4 import datetime
5
6
7 my_dag = DAG(
8     dag_id = 'my_very_first_dag',
9     description = 'My first DAG created with DataScientest',
10    tags = ['tutorial', 'datascientest'],
11    schedule_interval = None,
12    default_args = {
13        'owner': 'airflow',
14        'start_date': days_ago(2),
15    }
16 )
17
18 def print_date_and_hello():
19     raise TypeError('This will not work')
20     print(datetime.datetime.now())
21     print('Hello from Airflow')
22
23 def print_date_and_hello_again():
24     print('Hello from Airflow again')
25
26
27 my_task = PythonOperator(
28     task_id = 'my_very_first_task',
29     python_callable = print_date_and_hello,
30     dag = my_dag,
31 )
32
33 my_task2 = PythonOperator(
34     task_id = 'my_second_task',
35     python_callable = print_date_and_hello_again,
36     dag = my_dag
37 )
38
39 my_task >> my_task2
```

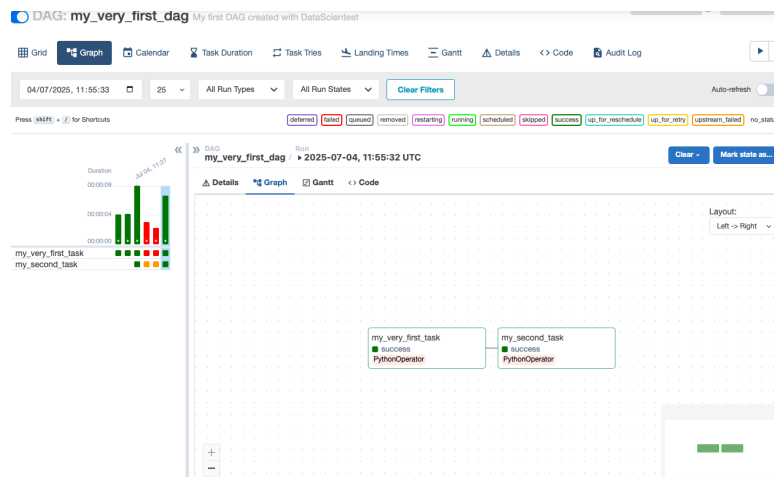
Run DAG again - first Task does not work (failed status) and second task due to dependency of first cannot begin (upstream_failed)



Simplifying DAGs:
Refactor DAG code

```
my_first_dag.py U X
datascientest-data-engineering > airflow > dags > my_first_dag.py

1  from airflow import DAG
2  from airflow.utils.dates import days_ago
3  from airflow.operators.python import PythonOperator
4  import datetime
5
6
7  with DAG(
8      dag_id = 'my_very_first_dag',
9      description = 'My first DAG created with DataScientest',
10     tags = ['tutorial', 'datascientest'],
11     schedule_interval = None,
12     default_args = {
13         'owner': 'airflow',
14         'start_date': days_ago(2),
15     }
16 ) as my_dag:
17
18     def print_date_and_hello():
19         print(datetime.datetime.now())
20         print('Hello from Airflow')
21
22     def print_date_and_hello_again():
23         print('Hello from Airflow again')
24
25
26     my_task = PythonOperator(
27         task_id = 'my_very_first_task',
28         python_callable = print_date_and_hello
29     )
30
31     my_task2 = PythonOperator(
32         task_id = 'my_second_task',
33         python_callable = print_date_and_hello_again
34     )
35
36     my_task >> my_task2
```



Application exercise:

Create `my_dag_of_the_morning.py` file

```
my_dag_of_the_morning.py U X
atascientest-data-engineering > airflow > dags > my_dag_of_the_m
1 from airflow import DAG
2 from airflow.utils.dates import days_ago
3 from airflow.operators.python import PythonOperator
4
5
6 with DAG(
7     dag_id = 'my_dag_of_the_morning',
8     description = 'DAG describing my morning routine',
9     tags = ['tutorial', 'datascientest'],
10    schedule_interval = None,
11    default_args = {
12        'owner': 'airflow',
13        'start_date': days_ago(2),
14    }
15 ) as my_dag:
16
17     def print_task(step):
18         print(step)
19
20     texts = [
21         'Put on trousers',
22         'Put on right sock',
23         'Put on left sock',
24         'Put on right trainer',
25         'Put on left trainer',
26         'Start moving'
27     ]
28
29     ids = [
30         'trousers',
31         'right_sock',
32         'left_sock',
33         'right_trainer',
34         'left_trainer',
35         'exit'
36     ]
37
38     tasks = []
39     for t, task_id in zip(texts, ids):
40         op = PythonOperator(
41             task_id = task_id,
42             python_callable = print_task,
43             op_kwargs = {'step': t},
44         )
45         tasks.append(op)
46
47     tasks[0] >> [tasks[1], tasks[3]]
48     tasks[1] >> tasks[2]
49     tasks[3] >> tasks[4]
50     [tasks[2], tasks[4]] >> tasks[5]
```

DAG: my_dag_of_the_morning DAG describing my morning routine

Schedule: none Next Run: none

Grid Graph Calendar Task Duration Task Tries Landing Times Gantt Details Code Audit Log

04/07/2025, 12:48:31 25 All Run Types All Run States Clear Filters Auto-refresh

Press **shift** + **/** for Shortcuts deferred failed queued removed restarting running scheduled skipped success up_for_reschedule up_for_retry upstream_failed no_status

The DAG visualization shows a timeline from 00:00:00 to 00:00:24. Tasks are represented by vertical bars: red for failed, orange for pending, green for successful, and blue for skipped. The tasks shown are trousers, right_sock, left_sock, right_trainer, left_trainer, and exit.

Task	Status
trousers	Failed
right_sock	Pending
left_sock	Pending
right_trainer	Pending
left_trainer	Pending
exit	Pending

my_dag_of_the_morning

Details Graph Gantt Code

DAG Runs Summary

Total Runs Displayed	5
Total success	2
Total failed	3
First Run Start	2025-07-04, 12:24:11 UTC
Last Run Start	2025-07-04, 12:48:31 UTC
Max Run Duration	00:00:24
Mean Run Duration	00:00:13
Min Run Duration	00:00:05

DAG Summary

Total Tasks	6
-------------	---

 DAGs Cluster Activity Datasets Security Browse Admin Docs 12:53 UTC AA									
☐	 	success	my_dag_of_the_morning	trousers ▼					
☐	 	success	my_dag_of_the_morning	right_sock ▼					
☐	 	success	my_dag_of_the_morning	right_trainer ▼					
☐	 	success	my_dag_of_the_morning	left_trainer ▼					
☐	 	success	my_dag_of_the_morning	left_sock ▼					
☐	 	success	my_dag_of_the_morning	exit ▼					

The screenshot displays the Dagster web interface for a DAG named 'my_dag_of_the_morning'. The top navigation bar includes tabs for Grid, Graph, Calendar, Task Duration, Task Trials, Landing Times, Gantt, Details, Code, and Audit Log. The main view shows a Gantt chart on the left and a DAG graph on the right. The Gantt chart displays the execution timeline of tasks: trousers, right_sock, left_sock, right_trainer, left_trainer, and exit. The DAG graph shows the flow of tasks: trousers (PythonOperator) leads to right_trainer and right_sock (both PythonOperators), which then lead to left_trainer and left_sock (both PythonOperators), finally leading to exit (PythonOperator). All tasks are marked as successful.